FusedLinearAttention: A Custom CUDA Kernel Fusing QKV Projection and Attention for Transformer Inference

Jnanasree Konda[1], Bhanuja Karumuru[1], and Rithwik Amajala[1]

[1] *Department of Electrical and Computer Engineering, New York University Tandon School of Engineering*

Abstract: Standard transformer implementations execute QKV projection and attention computation as two separate CUDA kernel launches, causing intermediate tensors to be written to and read back from GPU High Bandwidth Memory (HBM) between operations. We propose **FusedLinearAttention**, a custom CUDA kernel that computes QKV projection and attention in a single pass, eliminating this intermediate HBM roundtrip. Our claim is that the fused kernel reduces memory bandwidth consumption and kernel launch overhead compared to the standard unfused pipeline, while producing numerically identical outputs. We validate the approach on a transformer-based time-series forecasting model using the Electricity Transformer Dataset (ETTh1), profiling with NVIDIA NSight Systems to produce kernel-level timelines and memory bandwidth measurements.

Keywords: *CUDA kernel | attention fusion | HBM bandwidth | transformer inference | NSight profiling*

INTRODUCTION

Transformer models have become the dominant architecture for a wide range of tasks, yet their inference efficiency at the kernel level remains an active area of improvement. In a standard transformer implementation, the QKV projection step (three linear layers producing query, key, and value matrices) and the subsequent attention computation are executed as separate GPU operations. This separation means intermediate QKV tensors must be written to HBM after the projection step and read back again before attention begins — an unnecessary memory roundtrip that consumes bandwidth and adds kernel launch overhead.

FlashAttention [1] addresses the memory bottleneck *within* the attention computation by fusing the softmax and matrix multiplications into a single tiled kernel. However, it still assumes QKV tensors are already materialized in HBM before it begins. The step immediately before FlashAttention — the QKV projection itself — remains a separate, unfused operation.

We propose **FusedLinearAttention**, a custom CUDA kernel that fuses the QKV projection with the attention computation into a single kernel launch. Input activations are loaded from HBM once, QKV matrices are computed in on-chip shared memory and registers, attention is computed immediately without writing intermediate results back to HBM, and the final output is written to HBM once. This eliminates the intermediate memory roundtrip entirely and reduces the number of kernel launches from two to one.

RELATED WORK

Standard and Fused Attention Implementations

Standard PyTorch implements QKV projection and attention as separate operations. The `nn.Linear` layer and the attention mechanism each launch their own CUDA kernels, with intermediate results stored in HBM between them.

`torch.nn.functional.scaled_dot_product_attention` (SDPA) [2] is PyTorch's built-in fused attention kernel, which merges the softmax and matrix multiplications within attention. It does not fuse the preceding QKV projection.

FlashAttention-2 [1] is the state-of-the-art memory-efficient attention kernel, using tiling and recomputation to avoid materializing the full attention matrix. Like SDPA, it takes already-computed QKV tensors as input and does not address the projection step.

cuDNN attention provides a black-box fused attention implementation but is not transparent or customizable for research purposes.

NVIDIA Transformer Engine and the Remaining Gap

The closest existing work to our approach is NVIDIA Transformer Engine (TE) [5], a production library for accelerating transformer training and inference on Hopper-generation GPUs. TE provides a `LayerNormLinear` module that fuses LayerNorm and the QKV linear projection into a single kernel — a meaningful improvement over the standard PyTorch pipeline. Its `MultiheadAttention` module also exposes a `fuse_qkv_params` flag that consolidates the Q, K, and V weight matrices into a single parameter, reducing the number of matrix multiplications.

However, TE still performs a write to HBM between the projection step and the attention step. The `LayerNormLinear` kernel produces a materialized QKV tensor in HBM, which is then consumed by a downstream `DotProductAttention` kernel (backed by FlashAttention or cuDNN). The forward pass follows the pattern:

$$\texttt{Input} \xrightarrow{\texttt{LayerNormLinear}} \texttt{QKV} \xrightarrow{\texttt{HBM}} \texttt{QKV} \xrightarrow{\texttt{DotProductAttention}} \texttt{Out}$$

Kernel 1 (writes QKV to HBM) Kernel 2 (reads from HBM)

TE's primary performance gain comes from FP8 quantization during the projection step (exploiting Hopper Tensor Cores), not from eliminating the QKV-to-attention HBM write. Furthermore, TE's kernels are black-box compiled artifacts, making the fusion boundary opaque and non-customizable for research purposes.

Our work targets precisely the boundary that TE leaves open: the HBM write of QKV tensors between projection and attention. By computing QKV projections in on-chip shared memory and registers and immediately consuming them within the same kernel, **FusedLinearAttention** eliminates this roundtrip without requiring FP8

or Hopper-specific hardware. The result is a transparent, research-accessible kernel that demonstrates the bandwidth and latency gains available from closing this gap. This positions our contribution as complementary to both FlashAttention-2 (which fuses operations *within* attention) and TE (which fuses operations *before* attention) — our kernel fuses *across* the boundary between them.

Other Related Approaches

Recent work on fused QKV projection at the systems level includes the `qkv_fusion` kernel [6], which fuses Q, K, V projections using cuBLAS with Tensor Core support and reports a 2× speedup over the PyTorch baseline on NVIDIA H800 GPUs. However, this work fuses the three projection matmuls into one kernel, but still materializes QKV to HBM before attention begins. TensorRT-LLM's masked multi-head attention kernel [7] applies on-the-fly QKV bias addition, RoPE, and quantization fused into the attention kernel, but targets the inference-serving setting and is not exposed as a composable module. Our approach is differentiated by being (a) a single fused kernel spanning projection and attention, (b) transparent and customizable, and (c) framework-agnostic via `torch.utils.cpp_extension`.

PROPOSED APPROACH

The core idea is straightforward. In the standard pipeline, the forward pass involves two separate kernel launches: first a batched matrix multiplication to compute Q, K, and V from the input, then the attention computation over those materialized tensors. Our fused kernel collapses these into one:

1. Load input activations from HBM once into shared memory.
2. Compute Q, K, and V projections entirely within registers and shared memory, without writing them back to HBM.
3. Immediately compute the attention output using the in-register QKV values.
4. Write the final output to HBM once.

Tiling is used to handle sequences that exceed on-chip shared memory capacity. The kernel is bound to PyTorch via `torch.utils.cpp_extension`, making it a drop-in replacement within any PyTorch transformer model.

Correctness is verified by comparing numerical outputs against the standard PyTorch reference implementation, requiring maximum absolute difference below 10^{-4} across a range of sequence lengths.

EXPERIMENTAL PLAN

Dataset

We use the Electricity Transformer Dataset (ETTh1), a standard time-series forecasting benchmark containing 17,420 hourly electricity load measurements (∼3 MB). Its small size means GPU time is spent entirely on kernel experiments rather than data loading, which is ideal for a profiling-focused project.

Model

We implement a lightweight transformer forecasting model (either vanilla Transformer or PatchTST [3]) in PyTorch, trained on ETTh1. Our fused kernel is dropped in as a replacement for the standard attention block. End-to-end forecasting accuracy (MSE and MAE) is compared against the baseline to confirm the kernel produces identical results in a real training context.

Pipeline

1. **Baseline profiling.** Profile the standard unfused PyTorch pipeline (separate `nn.Linear` + attention) using NSight Systems to measure HBM bytes read/written, kernel launch count, and wall time per attention call.
2. **Kernel implementation.** Write the fused `.cu` kernel with tiling and shared memory management. Bind to PyTorch and verify numerical correctness across sequence lengths 64, 128, 256, 512, and 1024.
3. **Profiling comparison.** Re-profile with NSight Systems after substituting the fused kernel. Compare kernel timelines, HBM bandwidth utilization, SM occupancy, and kernel launch overhead against the baseline.
4. **End-to-end validation.** Run full ETTh1 training and evaluation with the fused kernel. Confirm MSE/MAE matches the baseline within numerical tolerance and report wall-time speedup.

Comparison Points

Method	QKV Fused	Attn Fused
Standard PyTorch		
PyTorch SDPA		✓
FlashAttention-2		✓
TE LayerNormLinear	Partial	✓
FusedLinearAttn (ours)	✓	✓

Table 1. *Fusion coverage across implementations. "Partial" for TE indicates QKV is fused with LayerNorm but still materialized to HBM before attention.*

Compute

All experiments run on a single Greene A100 node. FlashAttention-2 requires an Ampere-generation GPU, which the A100 satisfies. The ETTh1 dataset is negligible in size, so nearly all GPU time goes toward kernel development and profiling. We estimate approximately 5–8 A100 GPU-hours in total, covering baseline profiling, kernel iteration, correctness testing, and end-to-end model validation.

NOVEL CONTRIBUTIONS

1. **FusedLinearAttention kernel.** A custom CUDA kernel fusing QKV projection and attention into a single pass, eliminating the intermediate HBM roundtrip that all existing transparent implementations incur. Unlike NVIDIA Transformer Engine, which still writes QKV to HBM between its `LayerNormLinear` and `DotProductAttention` modules, our kernel spans this boundary in a single kernel launch.
2. **NSight kernel timeline analysis.** Kernel-level profiling demonstrating the reduction from two kernel launches to one, with quantitative HBM bandwidth and occupancy measurements.
3. **End-to-end validation on ETTh1.** Confirmation that the fused kernel is a numerically correct, drop-in replacement in a real forecasting task with measurable efficiency gains.

EXPECTED OUTCOMES

We expect the fused kernel to reduce HBM traffic per attention call compared to the unfused baseline, since intermediate QKV tensors no longer need to be written to and read from HBM. This should manifest as lower memory bandwidth utilization in NSight and reduced wall-clock time per attention call. The NSight kernel timeline will visually show the fused kernel as a single block where the baseline shows two sequential blocks with a gap between them.

The primary risk is kernel correctness and performance at larger sequence lengths, where tiling becomes necessary and shared memory management is more complex. We mitigate this by starting development at small sequence lengths and scaling up incrementally. A secondary risk is that the fused kernel, despite reducing HBM traffic, may underperform due to lower GPU occupancy if the kernel is too large to schedule efficiently. We will monitor occupancy via NSight and tune tile sizes accordingly. If the custom kernel does not outperform SDPA in wall time, we will report the bandwidth reduction as a standalone result alongside an analysis of the occupancy trade-off.

ACKNOWLEDGEMENTS

We thank the NYU High Performance Computing team for access to the Greene cluster and NVIDIA A100 nodes.

REFERENCES

[1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.

[2] PyTorch Contributors. `torch.nn.functional.scaled_dot_product_attention`. *PyTorch Documentation*, 2023.

[3] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. *ICLR*, 2023.

[4] A. Vaswani *et al.* Attention is all you need. *NeurIPS*, 2017.

[5] NVIDIA Corporation. Transformer Engine. https://github.com/NVIDIA/TransformerEngine, 2023.

[6] H. Chen. `qkv_fusion`: High-performance CUDA kernels for fused QKV projection with FlashAttention-2. https://github.com/hilaryKChen/qkv_fusion, 2023.

[7] NVIDIA Corporation. TensorRT-LLM: Masked multi-head attention kernel. https://github.com/NVIDIA/TensorRT-LLM, 2023.